
Chapter 15 Implementing a Collection Class

15.1 Simple `ArrayList`

- Adding and Printing 
- Thinking about Encapsulation 
- Dealing with the Middle of the List 
- Another Constructor and a Constant 
- Preconditions and Postconditions 

15.2 A More Complete `ArrayList`

- Throwing Exceptions 
- Convenience Methods 

15.3 Advanced Features

- Resizing When Necessary 
- Adding an Iterator 

Introduction

*In **Chapters 10**, **11**, and **14** we saw how to use various data structures that are part of the collections framework. As a Java programmer, you will find it useful to have these off-the-shelf solutions available to you, but you can learn a lot from examining how these structures are implemented. In this chapter we will explore in detail how to implement one such structure.*

*Our goal is to understand the `ArrayList` class that was described in detail in **Chapter 10**. Recall that `ArrayList` is a generic class that is best described as `ArrayList<E>`, where `E` is filled in with a specific element type. Because generic classes are not easy to implement, we will first explore the implementation of a class called `ArrayIntList` that can be used to store a list of simple integer values. There aren't a lot of applications for a simple `ArrayIntList` object, but the code we write for it will be similar to the code we need for the generic version. The chapter ends with a section that explores how to*

turn the more specific `ArrayList` into a generic `ArrayList<E>`.

15.1 Simple `ArrayIntList`

In this section, we will develop an initial version of the `ArrayIntList` class that contains appropriate fields for storing a list and a minimal set of operations for manipulating it.

Recall from [Chapter 8](#)  that there are two important perspectives of any class. The first is the external view that a client of the class will have. Clients in general don't want to understand all of the internal details of the class. They just want to know what it does. As a result, we want to have a clear specification for the client of what the `ArrayIntList` class is supposed to do. We think of this as a *contract* that we have with the client.

Contract

A clear specification that tells a client how an object behaves without describing the details of how it is implemented.

The second view is the internal view of the implementer. We have to figure out how to make the object work—how to satisfy the contract.

When we are thinking as an implementer, we will consider the nitty-gritty details of the innards of an object.

It is sometimes confusing to switch between these two different perspectives, but you will get better at it as you practice.

Adding and Printing

Let's start by developing a version of the class which has a single mutator method that will append values to the end of the list and a single accessor method that will display the contents of the list. Assume that the client code will look something like the following:

```
1 // A basic client that uses the ArrayIntList class.
2 public class Client1 {
3     public static void main(String[] args) {
4         // construct two lists
5         ArrayIntList list1 = new ArrayIntList();
6         ArrayIntList list2 = new ArrayIntList();
7
8         // add 1, 82, 97 to list1
9         list1.add(1);
10        list1.add(82);
11        list1.add(97);
12
13        // add 7, -8 to list2
```

```
14         list2.add(7);
15         list2.add(-8);
16
17         // report results
18         System.out.println("list1 = " + list1);
19         System.out.println("list2 = " + list2);
20     }
21 }
```

The preceding code will produce the following output:

```
list1 = [1, 82, 97]
list2 = [7, -8]
```

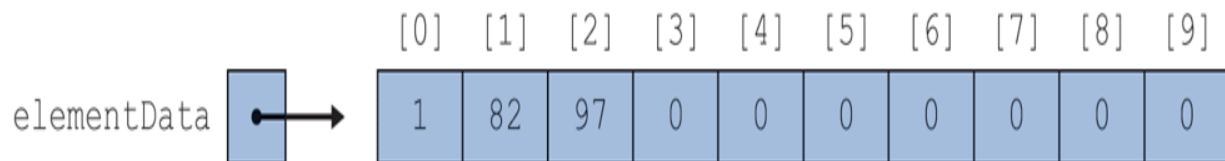
You must implement three methods in this program: the constructor, the `add` method, and the `toString` method that `println` will call. First you have to figure out what kind of fields you need. We are trying to emulate the `ArrayList` class, which is built on top of a simple array, so you should do the same thing for your `ArrayIntList` class. All of the examples of arrays in [Chapter 7](#) involved what could be called *filled arrays* in which each element of the array is in use. A filled array works fine in many applications, but this is not one of them. You are implementing a dynamic structure that will grow and shrink

as the client adds and removes values, so you don't want to be forced to use an array that is exactly the same size as your list.

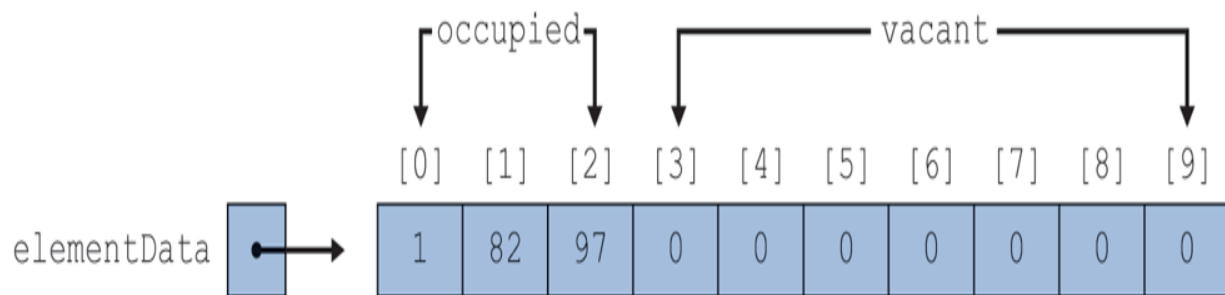
Instead, you should use an *unfilled array* in which some of the array elements are in use and some are not. This approach is similar to the way that a hotel is run. A hotel might have 100 guest rooms, but they don't all have to be occupied at once. There might be 90 rooms currently in use by customers and 10 vacant rooms that are available for later use.

This concept raises the question of how to distinguish between occupied cells and vacant cells in the array. One simple approach is to maintain a `size` variable that keeps track of the number of occupied cells and use the front of the array for cells that are currently in use and the back of the array for cells that are vacant. Unlike a hotel that has vacant occupied rooms interspersed, this kind of unfilled array groups all of the occupied cells together at the front of the array and all of the vacant cells at the end of the array.

For example, the sample client code at the beginning of this section constructs a list and adds the values `1`, `82`, and `97` to the list. That would mean that three of the array cells are occupied and any other cells are vacant. Suppose that you construct an array variable called `elementData` and you add these three values as the first three entries in the array:



While it's true that the first vacant cell has the value `0` stored in it, you wouldn't want to count on this being the case because the client might want to store the value `0` in the list. Without a `size` variable, you wouldn't know for sure whether this list ends with `97` or whether it ends with one of the zeros. With the `size` variable, you can keep track of exactly how many cells are occupied and you know that all others are vacant:



In the preceding example, the array has a length of 10. We refer to this length as the *capacity* of the list. In our hotel analogy, the hotel has 100 rooms, which means that it has the capacity to rent up to 100 rooms. But often the hotel is not filled to capacity. The same will be true for your list. The `size` variable keeps track of how much of the list is currently occupied whereas the capacity tells you how large the list can grow.

The fields you want to declare for your class are an array and a `size` variable:

```
public class ArrayIntList {  
    private int[] elementData;  
    private int size;  
    ...  
}
```

Your constructor should initialize these fields. The array in the preceding example had a capacity of 10, but let's increase it to 100:

```
public class ArrayIntList {  
    private int[] elementData;  
    private int size;  
    ...  
    public ArrayIntList() {  
        elementData = new int[100];  
        size = 0;  
    }  
    ...  
}
```

Recall from [Chapter 8](#)  that fields are initialized to the zero-equivalent for the type, which means that the `size` field will be

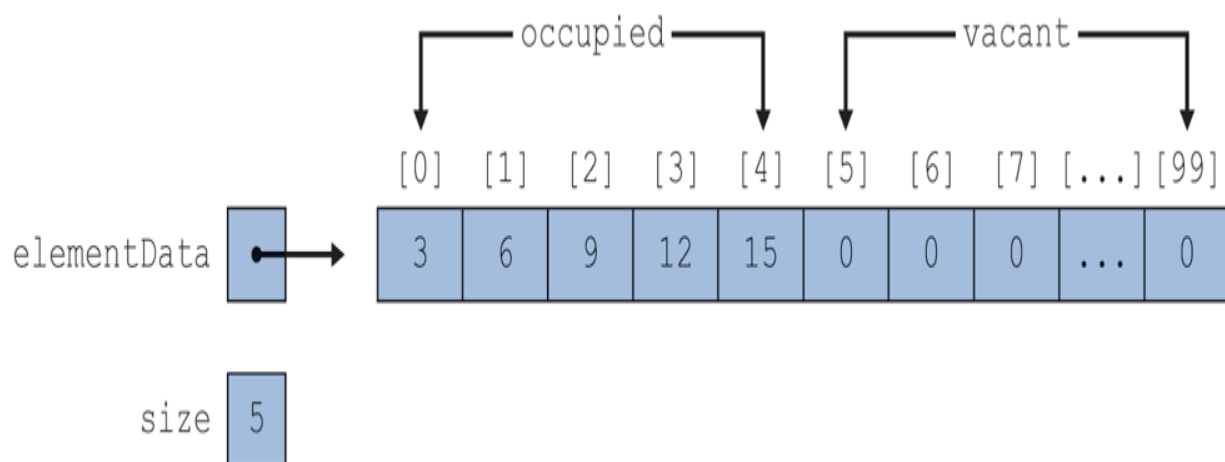
initialized to `0` with or without the line of code in the constructor.

Some Java programmers prefer to leave out the line of code because they are familiar with the default initialization. Others prefer to be more explicit about the initialization. This is somewhat a matter of personal taste.

Now that you have the fields and constructor defined, you can turn your attention to the `add` method. It takes an integer as a parameter. The idea is that you should append the given value to the end of the list. The method will look like this:

```
public void add(int value) {  
    ...  
}
```

This turns out to be a fairly simple method to implement. Suppose, for example, that the list stores the values `[3, 6, 9, 12, 15]`:



As indicated in the preceding figure, the current size of the list is 5. If you wanted to add the value 18 to the end of this list, you'd store it in the first vacant array element, which has index 5. In other words, when the list has a size of 5, you append the next value into index 5. And once you've added that value into index 5, then the list will have a size of 6 and the next vacant element will be at index 6.

Generally speaking, the size of the list will always match the index of the first vacant cell. This seems a bit odd, but it is a result of zero-based indexing. Because the first value is stored in index 0, the last value of a sequence of length n will be stored in index $n - 1$.

Therefore you should begin the `add` method by appending the given value at index `size`:

```
public void add(int value) {  
    elementData[size] = value;  
    ...  
}
```

Storing the value into the array isn't enough. You have to increment the `size` to keep track of the fact that one more cell is now occupied:

```
public void add(int value) {  
    elementData[size] = value;
```

```
        size++;  
    }
```

If your program did not increment `size`, it would keep adding values in the same index of the array. This would be like our hypothetical hotel checking every customer into the same hotel room because the hotel didn't keep track of which rooms are occupied.

It seems likely that this method will work, but so far the program doesn't have a good way of verifying it. You need some kind of method for displaying the contents of the list.

Chapter 7  explored the following code for printing the contents of an array of integers:

```
public static void print(int[] list) {  
    if (list.length == 0) {  
        System.out.println("[]");  
    } else {  
        System.out.print "[" + list[0];  
        for (int i = 1; i < list.length; i++) {  
            System.out.print(", " + list[i]);  
        }  
        System.out.println("]");  
    }  
}
```

The code involves a fencepost loop because we use commas to separate values. If there are n numbers, they will be separated by $n - 1$ commas. The code uses the classic fencepost solution of printing the first value before the loop. The `if/else` is used because there is a special case if the array is empty. This code can be fairly easily adapted to your `ArrayIntList` class. You have to convert the static method into an instance method. As a result, instead of having a parameter called `list`, you refer to the field called `elementData`. You also want to modify the code so that it uses the size of the list rather than the length of the array:

```
public void print() {  
    if (size == 0) {  
        System.out.println("[]");  
    } else {  
        System.out.print "[" + elementData[0];  
        for (int i = 1; i < size; i++) {  
            System.out.print(", " + elementData[i]);  
        }  
        System.out.println("]");  
    }  
}
```

But you don't want a method to print the list. If you want to make a truly useful class, you should provide flexible mechanisms that will work in a wide variety of situations. A better solution is to return a string representation of the list rather than printing it. As we saw in

Chapter 8 , Java has a standard name for such a method: `toString`. So you should rewrite the preceding method to construct and return a string representation of the list:

```
public String toString() {  
    if (size == 0) {  
        return "[]";  
    } else {  
        String result = "[" + elementData[0];  
        for (int i = 1; i < size; i++) {  
            result += ", " + elementData[i];  
        }  
        result += "];"  
        return result;  
    }  
}
```

Here is the complete class:

```
1 public class ArrayIntList {  
2     private int[] elementData;  
3     private int size;  
4  
5     public ArrayIntList() {  
6         elementData = new int[100];  
7         size = 0;
```

```
8      }
9
10     public void add(int value) {
11         elementData[size] = value;
12         size++;
13     }
14
15     public String toString() {
16         if (size == 0) {
17             return "[]";
18         } else {
19             String result = "[" + elementData[0];
20             for (int i = 1; i < size; i++) {
21                 result += ", " + elementData[i];
22             }
23             result += "[]";
24             return result;
25         }
26     }
27 }
```

This version of the class correctly executes the sample client program, producing the correct output.

Thinking about Encapsulation

When we declared the fields for `ArrayIntList`, we followed the usual convention of declaring them to be private. This can cause a problem for a client of the class. Suppose that the client has made a series of calls on the `add` method and wants to know how many elements are in the list. For example, the client might want to write code like the following:

```
Scanner input = new Scanner(new File("data.txt"));
ArrayIntList list = new ArrayIntList();
while (input.hasNextInt()) {
    list.add(input.nextInt());
}
System.out.println(list);
// report the size of the list, but how?
```

The information the client wants is stored internally in the field called `size`. But because the field is declared to be private, the client can't access the value.

There are several clumsy ways to solve this problem. One solution would be to change the field from private to public, but as we saw in [Chapter 8](#), making the field public breaks encapsulation and leaves the list object open to unwanted modifications. It's not a problem to allow the client to examine the value of the field, but suppose the client tried to set it to an illegal value, as in the following two examples:

```
list.size = -38;  
list.size = 5000;
```

Allowing the client to reach into the object opens the door for the client to put the object into a bad state.

Another solution would be to force the client to keep track of the size, but that is a rather silly solution. Why should both the object and the client keep track of this information? It puts an undue burden on the client and it leads to unnecessary duplication of effort.

The right solution is to keep the field private, but to introduce an accessor method that allows the client to examine the value of the `size` field:

```
public int size() {  
    return size;  
}
```

Then the client can write code like the following:

```
System.out.println("size = " + list.size());
```

This line of code looks fairly similar to code that accesses the field directly. The difference here is that we are calling a method called `size`. The method allows the client to examine the field without giving the client the ability to change the value of the field.

A client will also want the ability to access individual elements of the list. Internally, you access the list elements by referring to the array called `elementData`. Again, you don't want to break encapsulation by giving a client direct access to this field. Instead you can introduce another accessor method for examining the value stored at a given index:

```
public int get(int index) {  
    return elementData[index];  
}
```

Eventually, we want to make sure that this method can't be used by a client to access any of the vacant array elements (elements beyond the size of the list), but this simple version will do for now.

Dealing with the Middle of the List



So far we have explored how to add a value at the end of a list, but often a client is interested in dealing with values in the middle of the list. The client might want to search for the location of values or add them to or remove them from the middle of the list.

Let's begin with the task of searching for the location of a value in a list. The client could use the `size` and `get` methods to locate a value. But if it seems likely that a client will want to perform frequent searches for the locations of values, then it makes more sense to include the ability to do this as a method inside the class. That way, it will be readily available to all clients of the class.

In [Chapter 7](#), we wrote the following code to find the index of the first occurrence of a value in an array of integers:

```
public static int indexOf(int[] list, int target) {  
    for (int i = 0; i < list.length; i++) {  
        if (list[i] == target) {  
            return i;  
        }  
    }  
}
```

```
    }  
    return -1;  
}
```

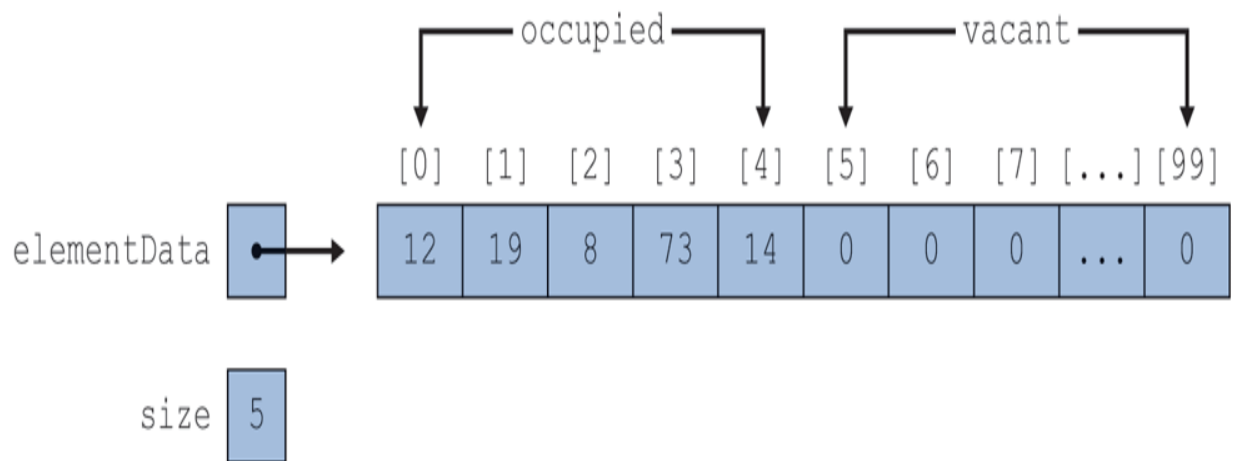
This method is fairly easily converted from a static method to an instance method, as you did with `print`. To achieve the conversion, replace the array parameter with references to the `elementData` field and, instead of using the length of the array as a loop bound, use the `size` field so that the method searches only elements of the array that are currently in use:

```
public int indexOf(int value) {  
    for (int i = 0; i < size; i++) {  
        if (elementData[i] == value) {  
            return i;  
        }  
    }  
    return -1;  
}
```

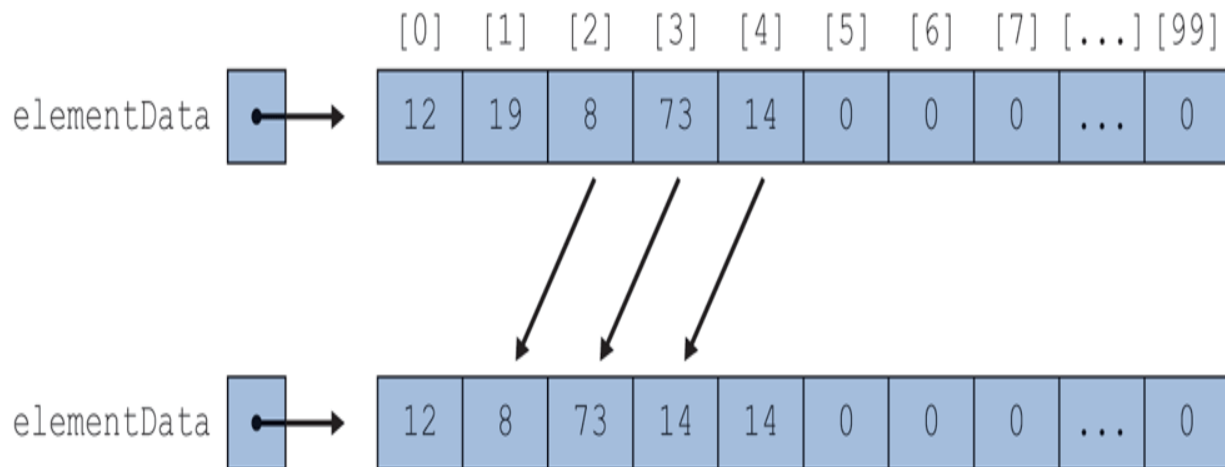
You haven't yet given the client the ability to remove a value from the list. Let's call the method `remove`. It should take as a parameter the index of the value to be removed:

```
public void remove(int index) {  
    ...  
}
```

Suppose, for example, that a list contains the five values `[12, 19, 8, 73, 14]`:



Suppose that you want to remove the value `19`, which is stored at index 1. This action will create a gap in the list unless you shift values over to fill in the gap. The three values that come after the value `19` each have to be shifted down by one position:



In order to shift the values over, you obviously need some kind of loop. You want to shift a value into index 1, another value into index 2, and another value into index 3. Why start at index 1? Because that's the target index, the index of the value you've been asked to remove. And why stop at index 3? Because that's when you run out of occupied cells to shift. The following code is a good first guess at the loop bounds:

```
for (int i = index; i < size; i++) {  
    ...  
}
```

We saw in [Chapter 7](#) that we can use code like the following to accomplish the shifting task:

```
for (int i = index; i < size; i++) {  
    elementData[i] = elementData[i + 1];  
}
```

```
}
```

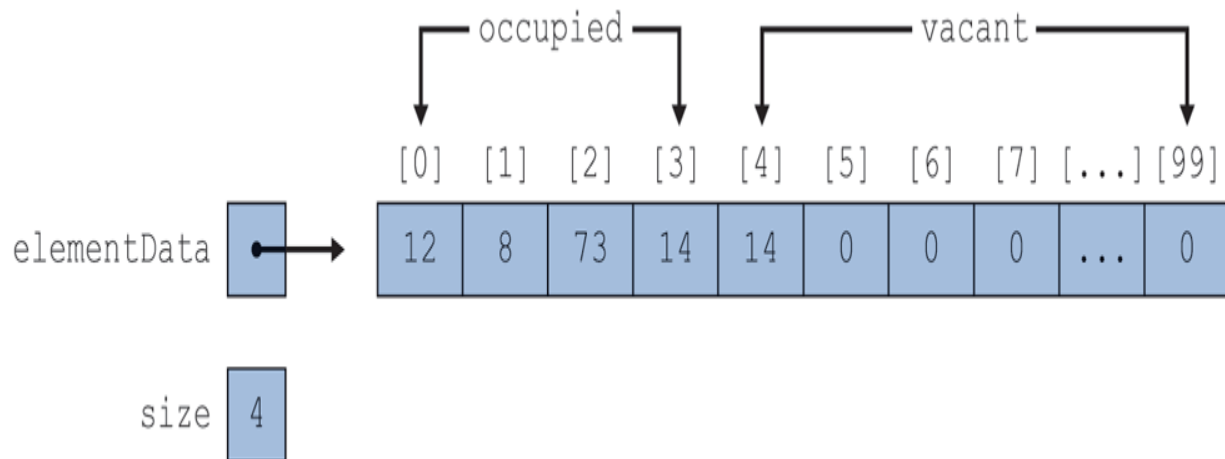
This code is almost correct. The problem is that the loop executes once too often. As we have noted, you want to shift values into indexes 1, 2, and 3. But when the `size` is 5, as in our example, the loop will do one extra shift, shifting a value into index 4. You want to stop the loop one step earlier so that it won't perform this final shift. You can accomplish that by subtracting one from the final loop bound:

```
for (int i = index; i < size - 1; i++) {  
    elementData[i] = elementData[i + 1];  
}
```

This modification almost completes the method. The only other detail you need to worry about is that once you have removed this value, you need to decrement the size of the list. Here is the complete method:

```
public void remove(int index) {  
    for (int i = index; i < size - 1; i++) {  
        elementData[i] = elementData[i + 1];  
    }  
    size--;  
}
```

Let's take a look at the final state of the list in our example. After we have removed the value at index 1, shifted three values left, and decremented the size, we end up with the following list:



Notice that the first cell among the vacant cells has the value 14 in it. You might imagine that you have to set it back to 0. In general, this isn't necessary for your `ArrayIntList`. Because the value is among the vacant cells, you know that it isn't a *real* value of the list. And if you ever need to use that cell again, you'll overwrite the 14 with some new value. It doesn't matter whether you overwrite a 14 or overwrite a 0. In fact, in the final version of our program, we'll make sure that a client can't ever see that extra value of 14. The situation is slightly different when you deal with the generic `ArrayList`, but we'll save that discussion for the end of the chapter.

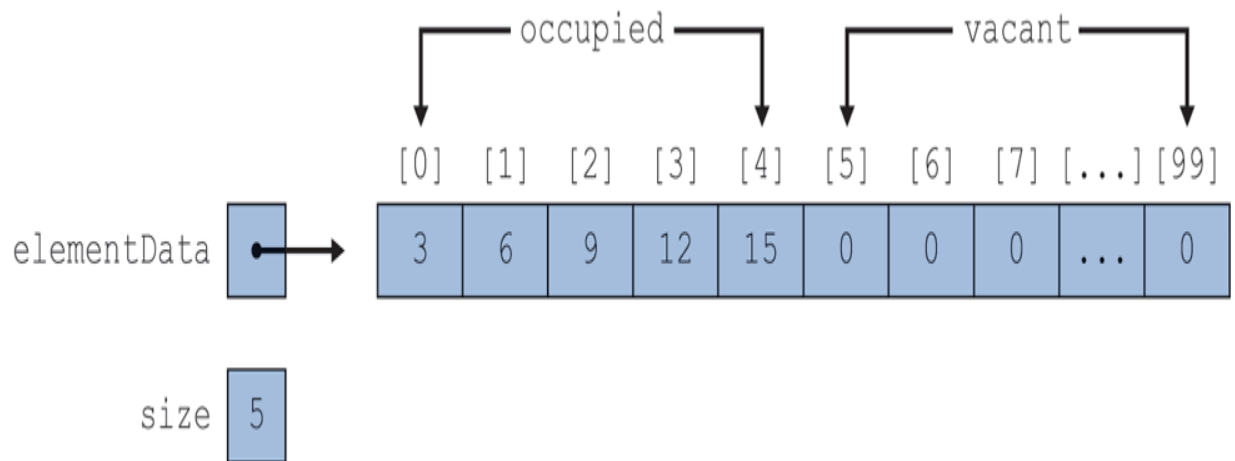
Another operation that a client is likely to want to perform is to add a value in the middle of a list. You already wrote a simple `add` method that appends at the end of the list:

```
public void add(int value) {  
    ...  
}
```

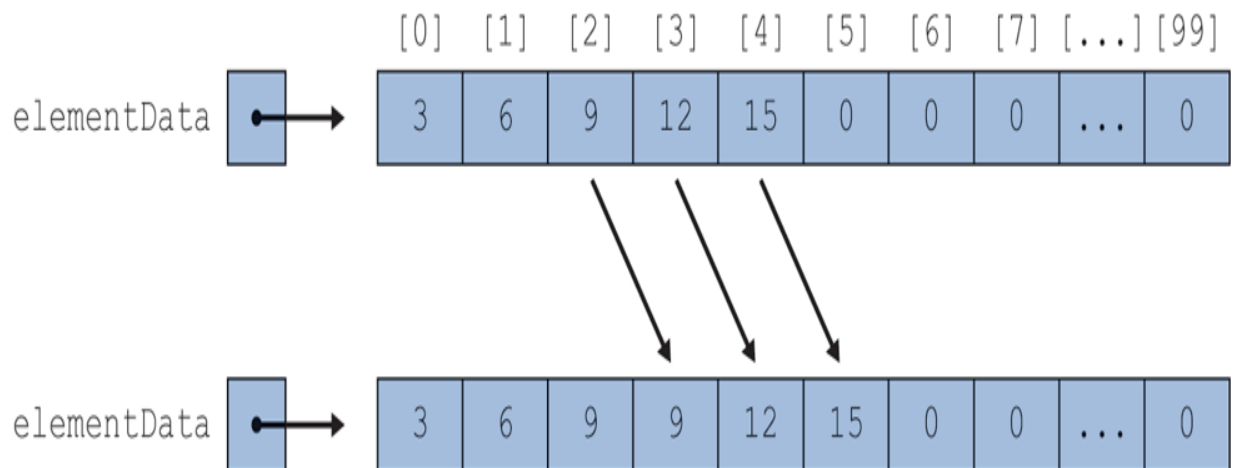
You can call the new method `add` as well, but it will have different parameters. You will still need to know the value to add, as with the appending method. But this new method will also need to know the index where the new value is to be added. Thus, it will look like the following:

```
public void add(int index, int value) {  
    ...  
}
```

Implementing this method is more complicated than appending a value at the end of the list because you have to shift values over to make room for the value to be inserted. Suppose, for example, that your list contains the five values `[3, 6, 9, 12, 15]`:



Suppose that you want to insert the value `7` at index 2. That insertion will require you to shift each of the values that are currently in indexes 2 through 4 to the right by one:



This insertion is similar to the remove operation. You want to deal with cells between the target index (where the new value is to be added) and the end of the sequence of occupied cells. Therefore, you know that the loop bounds will be something like the following:

```
for (int i = index; i < size; i++) {  
    ...  
}
```

We saw in [Chapter 7](#)  that you can shift values to the right by writing code like the following:

```
for (int i = index; i < size; i++) {  
    elementData[i] = elementData[i - 1];  
}
```

There are several problems with this code. First, it starts too early. In the example, you are trying to shift values so that you can add a new value at index 2. The first shift you want to do is to shift the value that is currently at index 2 into index 3. But the preceding loop will first shift the value at index 1 into index 2. The loop needs to start one index later:

```
for (int i = index + 1; i < size; i++) {  
    elementData[i] = elementData[i - 1];  
}
```

Even after you make this change, the loop bounds are still not correct. In the sample for a list of 5 elements, the final shift moved a value into the cell with index 5. The preceding code will stop the loop

once `i` is 4 (while it is still strictly less than the size). The loop needs to allow `i` to be equal to the size:

```
for (int i = index + 1; i <= size; i++) {  
    elementData[i] = elementData[i - 1];  
}
```

The final problem, as described in detail in [Chapter 7](#), is that this loop has to be run backward rather than forward. Otherwise you overwrite your list values with the value that is stored at the target index:

```
for (int i = size; i >= index + 1; i--) {  
    elementData[i] = elementData[i - 1];  
}
```

This modification completes the shifting, but you also have to store the new value at the target index now that there is room for it, and you have to increment the size of the list:

```
public void add(int index, int value) {  
    for (int i = size; i >= index + 1; i--) {  
        elementData[i] = elementData[i - 1];  
    }  
    elementData[index] = value;  
}
```

```
        size++;  
    }
```

Another Constructor and a Constant

The `ArrayIntList` class is shaping up nicely, but so far you have just a single constructor that constructs a list with a capacity of 100:

```
public ArrayIntList() {  
    elementData = new int[100];  
    size = 0;  
}
```

This constructor isn't very flexible. What if clients want to manipulate a list of 200 values? What are they supposed to do? You don't want to force them to rewrite the code just to use your class. That would be like having to open up a radio and rewire the insides just to change the volume or the station. If there is some value like the capacity that the client is likely to want to change, then you want to be sure to build in the flexibility to allow the client to do so.

You can accomplish this flexibility by changing the constructor to take a parameter that specifies the capacity of the list. You can then

use that value when you construct the array:

```
public ArrayIntList(int capacity) {  
    elementData = new int[capacity];  
    size = 0;  
}
```

This modification allows a client to write lines of code like the following:

```
ArrayIntList list1 = new ArrayIntList(200);
```

Unfortunately, if this is your only constructor, then the client loses the ability to write lines of code like the following:

```
ArrayIntList list2 = new ArrayIntList();
```

You could include both constructors, but then you have redundant code. As we saw in [Chapter 8](#) , you can avoid the duplication by having one constructor call the other. The constructor that takes the capacity is the more general constructor, so you can have the constructor that takes no arguments call it using the `this(...)` notation:

```
public ArrayIntList() {  
    this(100);  
}  
  
public ArrayIntList(int capacity) {  
    elementData = new int[capacity];  
    size = 0;  
}
```

You might wonder how Java can tell the two constructors apart. The answer is that they have different signatures. One constructor takes no arguments, whereas the second constructor takes an integer as an argument. When Java sees the call on `this(100)` in the first constructor, it knows that it is calling the second constructor because the call includes an integer value as a parameter.

Another improvement we can make is to introduce a constant for the rather arbitrary value of `100`:

```
public static final int DEFAULT_CAPACITY = 100;
```

It is a good idea to make this constant public because the client might want to be able to refer to the value to know what capacity a list has if the client doesn't specify a specific value to use. You would then rewrite the first constructor to use the constant instead of the specific value:

```
public ArrayList() {  
    this(DEFAULT_CAPACITY);  
}
```

Preconditions and Postconditions



We are almost ready to put all of these pieces together into a complete class. But before we do so, we should consider the issue of documentation. When you document the class, you want to think in terms of important information that should be conveyed to the client of the class. You want to describe what each method does and you want to describe any limitations of each method. This is a great place to use preconditions and postconditions, as described in [Chapter 4](#).

Recall from [Chapter 4](#) that preconditions are assumptions the method makes. They are a way of describing any dependencies that

the method has (“this has to be true in order for me to do my work”). Also, recall that postconditions describe what the method accomplishes, assuming that the preconditions are met (“I’ll do this as long as the preconditions are met”). The combination of preconditions and postconditions is a way of describing the contract that a method has with the client.

Consider, for example, the `get` method. It is a fairly simple method that allows the client to access individual elements of the list:

```
public int get(int index) {  
    return elementData[index];  
}
```

This method makes sense only if the value of `index` is in the range of occupied cells. In other words, it has to be greater than or equal to `0` and it has to be less than the size of the list. We can describe these constraints in a precondition comment:

```
// pre : 0 <= index < size()  
public int get(int index) {  
    return elementData[index];  
}
```

The postcondition should describe what the method accomplishes, assuming that the precondition is met. You can do that with a short description of the method's behavior:

```
// pre : 0 <= index < size()
// post: returns the integer at the given index in the list
public int get(int index) {
    return elementData[index];
}
```

There are quite a few preconditions for the methods you have written. All of the methods that take an index as a parameter assume that it is a legal index. The constructor that takes a capacity can't work with a negative value. And there is another important precondition lurking out there. Remember that your list has a fixed capacity. What happens if the client calls the `add` method so many times that the capacity is exceeded? You should warn the client about that scenario to make the contract clear.

In the next section we'll examine how to handle bad values more directly, but for now we'll settle for documenting the various preconditions.

It is worth noting that the method comments for this class do not contain comments about implementation. For example, we wouldn't mention that the `remove` method shifts values in the array. That's a detail of the implementation that won't generally interest the client. If

you think that the implementation is complicated enough to deserve comments, then include those comments with the code itself inside the method. The comment on the method itself should be written from the client perspective, not from the perspective of the implementer.

Professional Java programmers write their comments in Javadoc format so that they can be extracted from the actual program files and published as html files. If you've been referring to the Java API documentation, then you've been reading such comments. We don't use Javadoc format for our comments, but you can read about it in [Appendix B](#).

Here is a complete version of the `ArrayIntList` class incorporating all of the pieces discussed in this section:

```
1 // Class ArrayIntList can be used to store a list of
  integers.
2
3 public class ArrayIntList {
4     private int[] elementData; // list of integers
5     private int size;          // number of elements in the
  list
6
7     public static final int DEFAULT_CAPACITY = 100;
8
9     // post: constructs an empty list of default capacity
```

```
10     public ArrayIntList() {
11         this(DEFAULT_CAPACITY);
12     }
13
14     // pre : capacity >= 0
15     // post: constructs an empty list with the given
capacity
16     public ArrayIntList(int capacity) {
17         elementData = new int[capacity];
18         size = 0;
19     }
20
21     // post: returns the current number of elements in the
list
22     public int size() {
23         return size;
24     }
25
26     // pre : 0 <= index < size()
27     // post: returns the integer at the given index in the
list
28     public int get(int index) {
29         return elementData[index];
30     }
31
32     // post: returns comma-separated, bracketed version of
list
33     public String toString() {
```

```
34         if (size == 0) {
35             return "[]";
36         } else {
37             String result = "[" + elementData[0];
38             for (int i = 1; i < size; i++) {
39                 result += ", " + elementData[i];
40             }
41             result += "]";
42             return result;
43         }
44     }
45
46     // post : returns the position of the first occurrence of
the
47     //         given value (-1 if not found)
48     public int indexOf(int value) {
49         for (int i = 0; i < size; i++) {
50             if (elementData[i] == value) {
51                 return i;
52             }
53         }
54         return -1;
55     }
56
57     // pre : size() < capacity
58     // post: appends the given value to the end of the list
59     public void add(int value) {
60         elementData[size] = value;
```

```

61         size++;
62     }
63
64     // pre : size() < capacity && 0 <= index <= size()
65     // post: inserts the given value at the given index,
shifting
66     //         subsequent values right
67     public void add(int index, int value) {
68         for (int i = size; i >= index + 1; i--) {
69             elementData[i] = elementData[i - 1];
70         }
71         elementData[index] = value;
72         size++;
73     }
74
75     // pre : 0 <= index < size()
76     // post: removes value at the given index, shifting
77     //         subsequent values left
78     public void remove(int index) {
79         for (int i = index; i < size - 1; i++) {
80             elementData[i] = elementData[i + 1];
81         }
82         size--;
83     }
84 }

```

15.2 A More Complete

`ArrayIntList`

In this section, we will extend the `ArrayIntList` class to throw exceptions when preconditions are violated and to include a more inclusive set of methods that parallel the methods available for the built-in `ArrayList<E>`.

Throwing Exceptions



In the previous version of `ArrayIntList`, we documented the various preconditions of the methods. It is a good practice to clearly specify the contract with potential clients, but you shouldn't assume that clients will always obey the preconditions. As we saw in [Chapter](#)

4 , the convention in Java is to throw an exception when a precondition is violated.

For example, consider the constructor that takes a capacity as a parameter:

```
// pre : capacity >= 0
// post: constructs an empty list with the given capacity
public ArrayIntList(int capacity) {
    elementData = new int[capacity];
    size = 0;
}
```

The precondition indicates that the capacity is not supposed to be negative. You can go further than just documenting it by adding code to throw an exception. In this case, it is appropriate to throw an

`IllegalArgumentException`:

```
if (capacity < 0) {
    throw new IllegalArgumentException();
}
```

You have the option to include a string that will be displayed with the exception. In this case, it would be useful to tell the client the value that was being passed for `capacity`:

```
if (capacity < 0) {  
    throw new IllegalArgumentException("capacity: " +  
    capacity);  
}
```

You should also mention the exception in the comments for the method. It is important to mention exactly which type of exception will be thrown and under what circumstances. Here, then, is the final version of the method with the extra code to throw an exception when the precondition is violated:

```
// pre : capacity >= 0 (throws IllegalArgumentException if not)  
// post: constructs an empty list with the given capacity  
public ArrayIntList(int capacity) {  
    if (capacity < 0) {  
        throw new IllegalArgumentException("capacity: " +  
        capacity);  
    }  
    elementData = new int[capacity];  
    size = 0;  
}
```

In order to construct an exception object to be thrown, you have to decide what type of exception to use. All of the examples we have seen so far have constructed exceptions of type

`IllegalArgumentException`. This is a kind of generic exception that is

used to indicate that some value passed as an argument was not legal. But there are other exception types that you can use. The convention in Java is to pick the most specific exception that you can. **Table 15.1**  lists of some of the most common exception types.

Table 15.1 Common Exception Types

`NullPointerException` and `ArrayIndexOutOfBoundsException` are thrown by Java automatically, so you don't generally write code to specify those exceptions. Java programmers often use the other four exceptions, however.

Both of the `add` methods in your `ArrayIntList` class have a precondition that the size of the list must be strictly less than the capacity. Otherwise you will attempt to store a value in an array index that doesn't exist. This is an appropriate situation for an `IllegalStateException`. It is not appropriate to add values to the list once it has reached its capacity. In this case, you want to let the client know that the problem doesn't come from the values passed as arguments, but rather, that the `add` method was called at an inappropriate time.

You could add code to each of the `add` methods to check for this exception, but to avoid redundancy, it is better to introduce a private method that each of the methods calls. You'll be introducing some other methods that add more than one value at a time, so it will be useful to write the private method in a fairly flexible way. You can

write a method that takes a required capacity as a parameter and that checks to make sure that the array has that capacity. In other words, you'll tell the method, "I need room for this many elements," and it will test whether the array has that much room. If it does not, the method throws an `IllegalStateException`:

```
// post: checks that the underlying array has the given
capacity,
//      throwing an IllegalStateException if it does not
private void checkCapacity(int capacity) {
    if (capacity > elementData.length) {
        throw new IllegalStateException("exceeds list
capacity");
    }
}
```

You can add the following line of code to each of the `add` methods to check that the array has the capacity to add one more value to the list:

```
checkCapacity(size + 1);
```

You also need to modify the comments to indicate that this exception is thrown. For example, here is the rewritten appending `add` method:

```
// pre : size() < capacity (throws IllegalStateException if
not)

// post: appends the given value to the end of the list

public void add(int value) {
    checkCapacity(size + 1);
    elementData[size] = value;
    size++;
}
```

Several methods specify the index of a value. The `get` method, for example, is supposed to return the value at a particular index. If you provide a value that is outside the bounds of the underlying array, then the method will throw an `ArrayIndexOutOfBoundsException`. This exception is better than nothing, but it doesn't cover all cases. For example, your list might have a capacity of 100 but have only 10 values stored in it. The client might try to access a value at index 10 or 11 or 50, and your method will return a result when you'd prefer it to throw an exception to let the client know that the index is not legal for a list of 10 elements.

In this case, it is better to throw an `IndexOutOfBoundsException` and to include the illegal index to let the client see exactly what bad value was passed as a parameter. There are multiple methods that refer to index values, so it is again useful to introduce a private method that can be called by each method:

```
// post: throws an IndexOutOfBoundsException if the given index
is
//      not a legal index of the current list
private void checkIndex(int index) {
    if (index < 0 || index >= size) {
        throw new IndexOutOfBoundsException("index: " + index);
    }
}
```

This private method will properly throw an exception for `get` and `remove`. It isn't quite the right test to use for the `add` method that takes an index as a parameter, because `add` has a slightly different precondition than the other methods. Normally, it doesn't make sense to refer to a value at index `size` because that index is beyond the last value stored in the list. For example, if the list stores 10 values, then you don't want to call `get` or `remove` for a nonexistent value at index 10. But for `add`, it makes sense to refer to this index. If there are currently 10 values in the list, then you would be allowed to add a new value at index 10 because it is a new value not currently in the list. Instead of calling the private method, the `add` method will have its own code for checking the index and throwing an `IndexOutOfBoundsException`.

There is a further benefit to adding all of this code to throw exceptions: Adding the code will ensure that your object is never in a corrupt state. It will always satisfy certain data invariants. (Data

invariants were described in [Chapter 8](#).) In particular, you know that

- `size <= elementData.length` always (your calls on `checkCapacity` make sure of that)
- the array elements that you examine are always in the range of 0 to `size - 1` (your calls on `checkIndex` make sure of that)
- your code will never generate an `ArrayIndexOutOfBoundsException`

It is extremely useful to know that your class will always satisfy these constraints. This knowledge is an added benefit of encapsulation. If a client could reach in and change the `size` or `elementData` fields, then you couldn't make these kind of guarantees.

Convenience Methods

The built-in `ArrayList<E>` class has many other methods besides the ones we have implemented so far, so it makes sense to go ahead and add some of those methods to your `ArrayIntList` class. These methods are largely for the convenience of the client, since you could achieve the same functionality by calling existing methods on the list.

Your class has a method called `indexOf` that can be used to search for the location of a value in the list. Sometimes, though, a client just wants to ask the simpler question of whether a particular value

appears in the list somewhere. The convention in Java is to use a method called `contains` for this purpose. It has a `boolean` return type:

```
public boolean contains(int value) {  
    ...  
}
```

How do you write this method? You don't want to duplicate the code that you included in `indexOf`, so instead you'll call `indexOf`.

Remember that it returns a value of `-1` if the value is not found, so you can test whether or not `indexOf` returned an index that is greater than or equal to `0`:

```
public boolean contains(int value) {  
    if (indexOf(value) >= 0) {  
        return true;  
    } else {  
        return false;  
    }  
}
```

This version violates Boolean Zen, which was covered in [Chapter 5](#). You can simply return the value of the expression, rather than including it in an `if/else` statement:

```
return indexOf(value) >= 0;
```

Thus, the method ends up being a single line of code:

```
public boolean contains(int value) {  
    return indexOf(value) >= 0;  
}
```

Another common method in the Java Collections Framework is called `isEmpty`. It returns a `boolean` value indicating whether or not the list is empty. This is another method that can be written concisely as a one-line method using the value of the `size` field:

```
public boolean isEmpty() {  
    return size == 0;  
}
```

Again, you don't need an `if/else` statement; you can simply return the value of the Boolean expression.

So far you have included methods to add values to and remove values from the list, but sometimes you simply want to replace the value at a certain location with some new value. This operation is referred to as the `set` method and is easy to implement. You have to

remember to include a call on `checkIndex` because the method has a precondition that is similar to that of `get` and `remove`:

```
public void set(int index, int value) {  
    checkIndex(index);  
    elementData[index] = value;  
}
```

Providing this method is important because it allows a client to avoid unnecessary inefficiency. For example, suppose that the list has 10,000 values stored in it and the client wants to replace the first value with something new. The `set` method does it very quickly. The alternative would be to call `remove`, which would shift over the other 9999 values, and then to call `add`, which would shift the other 9999 values back to where they were earlier. Using `set` is much more efficient.

You might think that the introduction of the `set` method breaks encapsulation, because it allows a client to change a value in the array. But it doesn't really break encapsulation because you force the client to get this access by calling a method. This fact allows your method to be in control of the client request. So, for example, you can call the `checkIndex` method to ensure that the array index is legal.

Another common operation that a client might wish to perform is to empty the structure of all values. This is referred to as the `clear` operation and can be accomplished quickly by resetting the `size` field back to its original value of `0`:

```
public void clear() {  
    size = 0;  
}
```

You might imagine that you have to reset all of the array elements back to `0`, but that's not necessary. You have written the code in such a way that a client can only get access to array elements `0` through `size - 1`. When `size` is reset to `0`, the client can't access any of the array elements. The only way those array elements will be used again is if the client makes calls on the `add` method, in which case the old values will be overwritten with new values supplied by the client. But just as with the `remove` method, you will find at the end of the chapter that you solve this problem slightly differently for the generic `ArrayList` class.

The last method we will add is a “bulk add” method called `addAll` that adds all of the values from a second `ArrayIntList`. It may seem a little odd to have one `ArrayIntList` deal with another `ArrayIntList`, but this actually happens fairly often. The idea is that the first `ArrayIntList` is supposed to add all of the values from the second

`ArrayIntList`, which means that the header for the method looks like the following:

```
public void addAll(ArrayIntList other) {  
    ...  
}
```

You can call the appending `add` method to add values to the list. You just need a loop that iterates over the values in the second list. You also need to call your `checkCapacity` method to make sure that the array has sufficient capacity to store these new values:

```
public void addAll(ArrayIntList other) {  
    checkCapacity(size + other.size);  
    for (int i = 0; i < other.size; i++) {  
        add(other.elementData[i]);  
    }  
}
```

This code refers to `other.size` and `other.elementData`, which you might imagine would generate an error because they are private fields. In fact, the code does not generate an error. Recall from the [Chapter 8](#) examples that the word “private” means that it is “private to the class.” This is not the way that we as humans understand the meaning of private. (If something is private to me,

then it shouldn't be available to other humans.) But in Java, one `ArrayList` object can access private elements of another `ArrayList` object because they both belong to the same class.

You can find the complete third version of the `ArrayList` class on the web page <http://buildingjavaprograms.com>.

15.3 Advanced Features

In this section, we explore adding some extra advanced functionality to the `ArrayIntList` class. First we will see how to implement the class so that it has no fixed capacity and will grow larger if necessary. Then we will learn how to provide an iterator over the list.

Resizing When Necessary

The built-in `ArrayList<E>` class has a notion of capacity, as does our `ArrayIntList` class. But instead of throwing an exception when the capacity is exceeded, the class creates a larger array. In other words, its capacity grows as needed to accommodate the addition of new values to the list.

It isn't generally easy to make an array bigger. Java doesn't allow you to stretch an array that was constructed previously. Instead, you have to allocate a brand-new array and copy values from the old array to the new array. An analogy would be the way shops and other businesses work in the real world. If you need some extra space for your store, you can't generally break down the wall and grab some of the space from the store next door. More often, you have to relocate your store to a larger space.

Obviously you don't want to construct a new array too often. For example, suppose you had space for 1000 values and found you needed space for one more. A poor solution would be to allocate a new array of length 1001 and copy the 1000 values over. Then, if you find you need space for one more, you could make an array that is 1002 in length and copy the 1001 old values over. This kind of growth policy would be very expensive.

A better idea would be to double the size of the array when you run out of space. If you have filled up an array of length 1000, you double its size to 2000 when the client adds something more. That particular call on `add` is expensive because it has to copy 1000 values from the old array to the new array. But you won't need to copy again for a while. You can add another 999 values before you'd need extra space. As a result, we think of the expense as being spread out or *amortized* over all 1000 calls on `add`. When the cost is spread out over 1000 adds, it is fairly low (a constant).

The built-in `ArrayList` class does something similar. The documentation is a little coy about this: "The details of the growth policy are not specified beyond the fact that adding an element has constant amortized time cost." If you look at the actual code, you'll find that it increases the capacity by 50% each time (a multiplier of 1.5).

So how do you add this functionality to your `ArrayIntList` class? You included a method called `checkCapacity` that throws an exception if

the array isn't big enough. You can simply replace this method with a new method that makes the array larger if necessary:

```
public void ensureCapacity(int capacity) {  
    if (capacity > elementData.length) {  
        int newCapacity = elementData.length * 2 + 1;  
        if (capacity > newCapacity) {  
            newCapacity = capacity;  
        }  
        int[] newList = new int[newCapacity];  
        for (int i = 0; i < size; i++) {  
            newList[i] = elementData[i];  
        }  
        elementData = newList;  
    }  
}
```

This version of the method works, but you can get a slight improvement by calling a built-in method called `Arrays.copyOf` that returns a copy of an array. It has the same functionality as the preceding code, but it is likely to run faster because this operation is what is known as a *block copy* operation that can be optimized to run faster. Thus, the method can be rewritten as follows:

```
public void ensureCapacity(int capacity) {  
    if (capacity > elementData.length) {
```

```
        int newCapacity = elementData.length * 2 + 1;
        if (capacity > newCapacity) {
            newCapacity = capacity;
        }
        elementData = Arrays.copyOf(elementData, newCapacity);
    }
}
```

The `checkCapacity` method was declared to be private, but the preceding method has been declared to be public, because a client might also want to make use of that method. For example, if the client recognizes that the capacity needs to be significantly increased, then it is useful to be able to call this method to resize it once rather than resizing several times. The `ArrayList<E>` class has this method available as a public method.

Adding an Iterator

In [Chapter 11](#) , we saw that it is common in the collections framework to use an iterator object to traverse a collection. Recall that an iterator should provide three basic operations:

- `hasNext()`, which returns true if there are more elements to be examined
- `next()`, which returns the next element from the list and advances the position of the iterator by one

- `remove()`, which removes the element most recently returned by `next()`

We will develop a class called `ArrayIntListIterator` that implements this functionality for an `ArrayIntList`. The usual convention in Java is to ask the collection to construct the iterator by calling the method `iterator()`:

```
ArrayIntList list = new ArrayIntList();  
// code to fill up list ...  
ArrayIntListIterator i = list.iterator();
```

Once we have obtained an iterator from the list, we can use its three methods to traverse the list. For example, the following program constructs an `ArrayIntList` and then computes the product of the list:

```
1 // A second client that uses the ArrayIntList class.  
2 public class Client2 {  
3     public static void main(String[] args) {  
4         // construct and print list  
5         int[] data = {13, 4, 85, 13, 40, -8, 17, -5};  
6         ArrayIntList list = new ArrayIntList();  
7         for (int n : data) {  
8             list.add(n);  
9         }
```

```

10      System.out.println("list = " + list);
11
12      // obtain an iterator to find the product of the
list
13      ArrayIntListIterator i = list.iterator();
14      int product = 1;
15      while (i.hasNext()) {
16          int n = i.next();
17          product *= n;
18      }
19      System.out.println("product = " + product);
20  }
21  }

```

The program produces the following output:

```

list = [13, 4, 85, 13, 40, -8, 17, -5]
product = 1562912000

```

Here is a variation that removes any `0` values from the list, computing the product of the nonzero values:

```

1  // A third client that uses the ArrayIntList class.
2  public class Client3 {

```

```

3      public static void main(String[] args) {
4          // construct and print list
5          int[] data = {5, 19, 0, 2, 4, 0, 13, 85, -8, 0,
23};
6          ArrayIntList list = new ArrayIntList();
7          for (int n : data) {
8              list.add(n);
9          }
10         System.out.println("list = " + list);
11
12         // use an iterator to find the product, removing
zeros
13         ArrayIntListIterator i = list.iterator();
14         int product = 1;
15         while (i.hasNext()) {
16             int n = i.next();
17             if (n == 0) {
18                 i.remove();
19             } else {
20                 product *= n;
21             }
22         }
23         System.out.println("list now = " + list);
24         System.out.println("product = " + product);
25     }
26 }

```

The program produces the following output:

```
list = [5, 19, 0, 2, 4, 0, 13, 85, -8, 0, 23]
list now = [5, 19, 2, 4, 13, 85, -8, 23]
product = -154523200
```

So how do you implement the `ArrayIntListIterator` class? The main function that the iterator performs is to keep track of a particular position in a list, so the primary field will be an integer variable for storing this position:

```
public class ArrayIntListIterator {
    private int position;
}

public ArrayIntListIterator(...) {
    position = 0;
    ...
}
...
}
```

Initially you start `position` at `0` so that it will refer to the first value in the list. The `hasNext` method is supposed to determine whether any values remain to iterate over. To do so, it will have to compare this position to the size of the list:

```
public boolean hasNext() {  
    // check position against size of the list  
}
```

To perform this comparison, the iterator needs to be able to find out the size of the list. To do so, the iterator will need to keep track of the list over which it is iterating. That means that you will need a second field and the constructor will have to be passed a reference to the list to iterate over:

```
public class ArrayIntListIterator {  
    private ArrayIntList list;  
    private int position;  
  
    public ArrayIntListIterator(ArrayIntList list) {  
        this.list = list;  
        position = 0;  
        ...  
    }  
  
    ...  
}
```

Using this field, you can now easily write the `hasNext` method:

```
public boolean hasNext() {  
    return position < list.size();  
}
```

What about the `next` method? It is supposed to return the next value from the list and then reset the position to be one later in the sequence, which you can accomplish by simply incrementing the value:

```
public int next() {  
    int result = list.get(position);  
    position++;  
    return result;  
}
```

But the method has an important precondition that you must consider. What if a client calls `next` when the iterator has run out of values to return? The method should throw an exception in that case. The convention in Java is to throw a `NoSuchElementException`:

```
public int next() {  
    if (!hasNext()) {  
        throw new NoSuchElementException();  
    }  
    int result = list.get(position);
```

```
    position++;  
    return result;  
}
```

The final operation performed by an iterator is the `remove` method.

The method is supposed to remove the most recent value that was returned by `next`. The `position` field keeps track of the next value to be returned by the iterator, so the value to be removed is at index

`position - 1`:

```
public void remove() {  
    list.remove(position - 1);  
    ...  
}
```

Keep in mind what happens when you ask the `ArrayList` to remove that value. All of the other values will be shifted one to the left in the list. That means that `position` will no longer be positioned at the next value in the list. That value has been shifted one to the left, so you have to decrement `position` to account for the shift that has taken place:

```
public void remove() {  
    list.remove(position - 1);  
    position--;  
}
```

```
    position--;  
}
```

This method also has an important precondition to consider. A client is supposed to call `next` before calling `remove`. One possibility is that the client will call `remove` before making any call on `next`. If that happens, it will be obvious from the fact that `position` will be zero. Another possibility is that the client will call `remove` twice in a row without calling `next` in between. That is not a legal operation either. You won't know just from looking at the value of `position` whether the client has violated this precondition.

In this case, you need an extra bit of state for the object. You need to keep track of whether it is currently legal to remove a value, so this is a good time to add a field. It will be of type `boolean` and you can call it `removeOK`. You can use this field to throw an exception if the precondition is violated. And once a call on `remove` has been performed, you have to remember that it is no longer legal to remove a value until `next` is called again:

```
public void remove() {  
    if (!removeOK) {  
        throw new IllegalStateException();  
    }  
    list.remove(position - 1);  
    position--;  
}
```

```
        removeOK = false;
    }
```

Notice that you throw an `IllegalStateException` because a call on the method is not appropriate if it is not okay to remove a value. You can then add code to the constructor to initialize this field to `false`, and you can reset it to `true` whenever `next` is called.

Here is the final class definition:

```
1  // Objects of this class can be used to iterate over an
2  // ArrayList and remove values from the list.
3
4  import java.util.*;
5
6  public class ArrayListIterator {
7      private ArrayList list;        // list to iterate
over
8      private int position;          // current list
position
9      private boolean removeOK;     // okay to remove now?
10
11     // post: constructs an iterator for the given list
12     public ArrayListIterator(ArrayList list) {
13         this.list = list;
14         position = 0;
15         removeOK = false;
```

```

16     }
17
18     // post: returns true if there are more elements left
19     public boolean hasNext() {
20         return position < list.size();
21     }
22
23     // pre : hasNext() (throws NoSuchElementException if
not)
24     // post: returns the next element in the iteration
25     public int next() {
26         if (!hasNext()) {
27             throw new NoSuchElementException();
28         }
29         int result = list.get(position);
30         position++;
31         removeOK = true;
32         return result;
33     }
34
35     // pre : next() has been called without a call on
remove
36     //      (throws IllegalStateException if not)
37     // post: removes the last element returned by the
iterator
38     public void remove() {
39         if (!removeOK) {
40             throw new IllegalStateException();

```

```
41     }  
42     list.remove(position - 1);  
43     position--;  
44     removeOK = false;  
45 }  
46 }
```

The program imports the `java.util` package because the class for one of the exceptions we want to throw, `NoSuchElementException`, comes from that package.

You also have to modify the `ArrayIntList` class. Remember that it needs to have a method called `iterator` that constructs an iterator, which means it will look like this:

```
public ArrayIntListIterator iterator() {  
    return new ArrayIntListIterator(...);  
}
```

So which list should you mention in the call on the `ArrayIntListIterator` constructor? The `ArrayIntList` is supposed to construct an iterator that is looking at itself. You can use the `this` keyword to say, “Construct an iterator that is iterating over me”:

```
public ArrayIntListIterator iterator() {  
    return new ArrayIntListIterator(this);  
}
```

You can find a complete listing of this fourth version of the `ArrayIntList` class along with the `ArrayIntListIterator` class on the web page for this textbook at <http://buildingjavaprograms.com>.

15.4 ArrayList<E>

In this section, we will explore how to convert the version of `ArrayIntList` from the previous section into a generic `ArrayList<E>`. To start, you can simply replace all occurrences of `ArrayIntList` with `ArrayList<E>` and change references to `int` that refer to values to `E`. Of course, there are other uses of `int` that specify capacity, size, and indexes, and these don't change.

This approach almost works, but there are a few places where you have to be careful. For example, when you define constructors, you don't use the generic `E` when you're naming the constructor. So the zero-argument constructor becomes:

```
public ArrayList() {  
    this(DEFAULT_CAPACITY);  
}
```

You also run into trouble in the second constructor. After you perform the simple substitution, you end up with the following method:

```
public ArrayList(int capacity) {  
    if (capacity < 0) {
```

```
        throw new IllegalArgumentException("capacity: " +  
capacity);  
    }  
    elementData = new E[capacity]; // illegal  
    size = 0;  
}
```

When you try to compile this version, you get an error indicating that you are not allowed to construct a generic array. This is a limitation of generic types. You can construct an array of type `Object[]`, but not an array of type `E[]`. You can solve this problem by introducing a cast. Replace the following line of code:

```
elementData = new E[capacity];
```

In its place, include this line of code:

```
elementData = (E[]) new Object[capacity];
```

This new version compiles, but it generates a warning about using unsafe or unchecked types. There's no real way to get around this because Java won't let you construct a generic array. That means you can ignore this warning. In this case, it is good to include an *annotation* for the method to indicate that you don't want it to generate the warning for this method:

```
@SuppressWarnings("unchecked")  
public ArrayList(int capacity) {  
    ...  
}
```

Annotations are special Java syntax that mark metadata on a class or method. They give instructions to the compiler. This particular annotation is a way of telling the compiler, “Don’t generate the warning for this method because I’m aware of the problem already.” There are many different kinds of annotations that you can include in your Java programs, but we won’t take time to explore them in this book.

There is also a problem with the `indexOf` method. After you perform the substitution of `E` for `int`, here is the result:

```
public int indexOf(E value) {  
    for (int i = 0; i < size; i++) {  
        if (elementData[i] == value) {  
            return i;  
        }  
    }  
    return -1;  
}
```

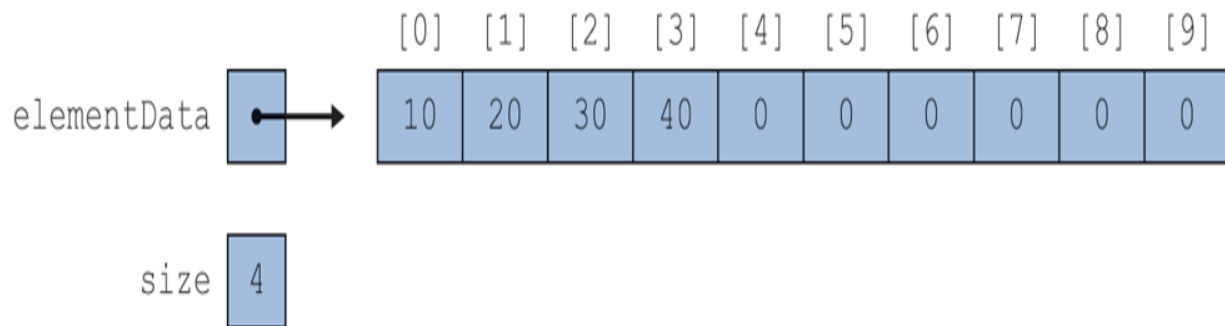
This version compiles and would work, but it has a very strict definition of equality. It would require that the actual object you are searching for appears in the list. More often, you want to use an `equals` comparison to see whether the value for which you are searching is equal to some value in the list. You'll want to replace the following bit of code from your method:

```
if (elementData[i] == value) {  
    ...  
}
```

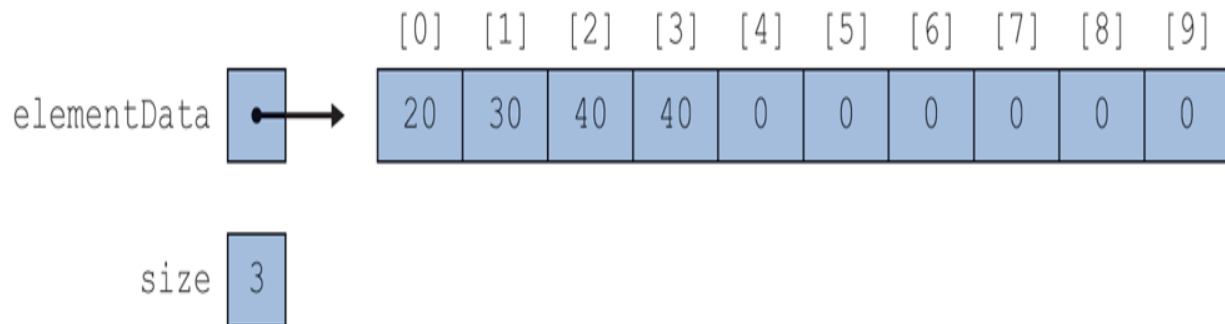
You can replace it with the following:

```
if (elementData[i].equals(value)) {  
    ...  
}
```

One final issue has to do with memory allocation. Consider what happens when a value is removed from the simple `ArrayIntList`. An array element that used to correspond to a list element is no longer being used. In general, this isn't a problem, and we haven't bothered to do any cleaning up afterward. For example, suppose that you have a list with a capacity of 10 and you store the values `[10, 20, 30, 40]` in the list. The array looks like this:



If you then remove the value at index `0`, you shift the other three values left and decrement `size`:



Notice that you now have two occurrences of `40` in the list. That isn't generally a problem because you know from the value of your `size` field that the `40` stored in index `3` isn't being used.

You can't be so cavalier when it comes to objects. We have to think about what is known as the *garbage collector*:

Garbage Collector

A process that is part of the Java Runtime Environment that periodically frees the memory used by objects that are no longer referenced.

In some programming languages, you have to explicitly destroy objects when you don't need to use them any longer. Java saves you the trouble of doing this. Instead, its garbage collector looks for objects that are no longer being used (i.e., objects that are no longer referenced). You want to make sure that your `ArrayList` doesn't interfere with what the garbage collector is trying to accomplish.

If your `ArrayList` is keeping a reference to some object that is no longer being used, then the garbage collector might not recognize that it can reclaim that space. So you have to explicitly set that array element back to `null`.

A direct translation of the code produces the following `remove` method:

```
public void remove(int index) {  
    checkIndex(index);  
    for (int i = index; i < size - 1; i++) {  
        elementData[i] = elementData[i + 1];  
    }  
}
```

```
        size--;  
    }
```

You need to add an extra line of code after the shifting code that sets the unused array element back to `null`:

```
public void remove(int index) {  
    checkIndex(index);  
    for (int i = index; i < size - 1; i++) {  
        elementData[i] = elementData[i + 1];  
    }  
    elementData[size - 1] = null;  
    size--;  
}
```

Similarly, the `clear` method needs to set all values to `null` before resetting the `size` field to `0`:

```
public void clear() {  
    for (int i = 0; i < size; i++) {  
        elementData[i] = null;  
    }  
    size = 0;  
}
```

One final change that you will see in this class is that the iterator class has been converted to what is known as an *inner class*:

Inner Class

A class declared inside another class. Objects of the inner class have access to the methods and fields of the outer class.

In other words, the structure becomes

```
public class ArrayList<E> {  
    ...  
    private class ArrayListIterator implements Iterator<E> {  
        ...  
    }  
}
```

The generic syntax can be a bit confusing with inner classes. We declare our inner `ArrayListIterator` class without an `<E>` type parameter, because the type `E` is already declared as part of the outer list class. But we do have to say that it implements `Iterator<E>` to match the interface from `java.util`. Accidentally declaring the

inner class as `ArrayListIterator<E>` actually creates a second generic type `E` and leads to confusing compiler errors.

Inner classes are normally declared to be private. Using an inner class for the iterator is the more usual approach. It allows you to eliminate the field that keeps track of the list. As we indicated in the definition box, when you declare an inner class, the instances of the inner class have access to the methods and fields of the instance of the outer class that constructed it. For example, the old version of the `hasNext` method refers to a field called `list` that was used to keep track of the list over which the iterator was iterating:

```
public boolean hasNext() {  
    return position < list.size();  
}
```

This code can be simplified. Because the iterator class is now an inner class, you can write the `hasNext` method in a simple way:

```
public boolean hasNext() {  
    return position < size();  
}
```

This method is calling the `size` method even though the iterator has no such method. Because it has no such method, it calls the

`ArrayList` method of the outer class.

This can cause problems in one part of the code. When you write the `remove` method for the iterator, you need to call the `remove` method of the list:

```
public void remove() {  
    if (!removeOK) {  
        throw new IllegalStateException();  
    }  
    remove(position - 1); // illegal  
    position--;  
    removeOK = false;  
}
```

Unfortunately, this version does not compile, because both the iterator and the list have a method called `remove`. One has no parameters and the other has one parameter, so you'd think that Java could tell the difference, but it can't. You have to use a special notation to make it clear that you want to refer to the outer object. You do that by referring to the `this` value of the outer class, which you refer to as `ArrayList.this`. Thus, you can rewrite the following line of code:

```
remove(position - 1);
```

Here is the new version of the code:

```
ArrayList.this.remove(position - 1);
```

Before we finish this exploration, we should consider at least briefly the issue of interfaces. As we saw in [Chapters 9](#) and [11](#), interfaces allow clients to describe variables in a more generic way. For example, suppose that you want to iterate over an `ArrayList` of `String` values. You'll replace the following line of code:

```
ArrayListIterator i = list.iterator();
```

Instead, you'll use the `Iterator<E>` interface:

```
Iterator<String> i = list.iterator();
```

In order for this code to compile, it is important for the `ArrayListIterator` class to implement the `Iterator` interface and for the return type for the `iterator()` method to use the interface rather than the class name. These changes have been incorporated into the final version of the class. It turns out that there are other interfaces to consider as well, but we will save that discussion until the end of the next chapter. At that time we will see a final version of the `ArrayList` class that is even closer to the built-in version.

You will find the complete code for the `ArrayList` class on our web page at <http://buildingjavaprograms.com>.

Chapter Summary

In this chapter, we implemented an array list class to store lists of integers.

Collection classes have two views: the external view seen by client code and the internal view seen by the implementer.

An array list uses an unfilled array and a `size` field in which the first *size* elements are meaningful and the rest are empty zeroes that are not considered to be part of the list. The entire array length represents its capacity for storing values.

When we add values to or remove values from the front or middle of an array list, we must shift the values right or left respectively to account for the newly added or removed element.

Our array list has preconditions that the client will not pass an illegal capacity on construction or illegal indexes when accessing elements. If the client does try to do so, Java will throw an exception.

Our subsequent versions of the list class resize to a larger capacity when the array becomes full.

The array list has an iterator for examining its elements in sequence.

Our array list of integers can be converted into a generic class that can store a list of any type of objects. The code is similar, but we must make a few changes—for example, when we construct arrays of type `E[]` or compare objects for equality.

An inner class is declared inside the braces of another (outer) class and has access to the state of an object of that outer class. Our final list iterator is an inner class.

Self-Check Problems

Section 15.1: Simple

ArrayIntList

1. What is the difference between an array list's size and its capacity? What is the relationship between the two values? (Is one always larger or smaller than the other, for instance?)
2. What fields must be included in the `ArrayIntList` class, and why is each field important? Would the class still work correctly if we removed any of these fields?
3. How would the output of the `Client1` program shown in this section change if each field from `ArrayIntList` were declared `static`?
4. In this version of the list class, what happens if the client adds too many values to fit in the array?
5. Why does the list class use a `toString` method rather than a `print` method?
6. We wrote the class to have public methods called `size` (to read the number of elements of the list) and `get` (to access the element value at a specific index). Why is this approach better than declaring the fields (such as `size`) `public`?
7. An element can be inserted at the beginning, middle, or end of an array list. Which of the three insertion points is the most computationally expensive, and why? Which is the most expensive location to remove an element from the list?

8. Write methods called `min` and `max` that return the smallest and largest values in the list respectively. For example, if a variable called `list` stores `[11, -7, 3, 42, 0, 14]`, the call of `list.min()` should return `-7` and the call of `list.max()` should return `42`. If the list is empty, the methods should throw an `IllegalStateException`.

Section 15.2: A More Complete

ArrayIntList

9. Describe the overall preconditions placed on the `list` class in this section. What assumptions do we make about how clients will use the list?
10. What is the purpose of the `checkIndex` method? Where is it called in the `list` class? Describe a way that the client can utilize an `ArrayIntList` that will be caught by `checkIndex`.
11. What is the purpose of the `checkCapacity` method? Where is it called in the `list` class? Describe a way that the client can utilize an `ArrayIntList` that will be caught by `checkCapacity`.
12. Once we check thoroughly for preconditions in the code, what data invariants can we now assume about the list?
13. Why do we bother to add the `contains`, `isEmpty`, and `remove` methods to the `list` class, when the client can already perform this same functionality with the `indexOf`, `size`, and `remove` methods, respectively?

Section 15.3: Advanced Features

14. When this new version of the class fills to its capacity, it resizes. How much does it grow? Why choose this growth rate, rather than increasing the capacity by a single element or other constant amount?
15. What is the benefit of adding an iterator to the list class?
16. What state does the array list iterator store?
17. How does the array list iterator know if there are more elements left to examine? What does it do if the client tries to examine a next element but there are none left to examine?
18. What is a precondition of the iterator's `remove` method? How does the iterator enforce this precondition, and what does it do if the precondition is violated?
19. Write a method called `sum` that returns the sum of all values in the list. For example, if a variable called `list` stores `[11, -7, 3, 42, 0, 14]`, the call of `list.sum()` should return `63`. If the list is empty, `sum` should return `0`.
20. Write a method called `average` that returns the average of the values in the list as a real number. For example, if a variable called `list` stores `[11, -7, 3, 42, 0, 14]`, the call of `list.average()` should return `10.5`. If the list is empty, `average` should return `0.0`.

Section 15.4: `ArrayList<E>`

21. What problem do we encounter when we try to construct an array of type `E`? How do we resolve this problem?
22. Since our list stores an unfilled array, the empty elements were filled with the value `0` when our array was full of integers. What value occupies the empty cells when our list stores values of type `E`?
23. What changes need to be made to the `indexOf` method to search for objects of type `E` in the new `list` class, and why are these changes necessary?
24. What is an annotation? How are annotations useful in writing our `ArrayList<E>` class?
25. Why is it important to set empty elements to `null` when we are clearing or removing from the list of type `E`, when we didn't need to clear out these elements in the previous `ArrayIntList`?
26. What is one benefit of making the list iterator into an inner class?

Exercises

Each of the following exercises is a method to be added to the `ArrayIntList` class from this chapter.

1. Write a method called `lastIndexOf` that accepts an integer as a parameter and returns the index in the list of the last occurrence of that value, or `-1` if the value is not found in the list. For example, if the list stores `[1, 18, 2, 7, 18, 39, 18, 40]`, then the last index of 18 is 6 and the last index of 3 is `-1`.
2. Write a method called `indexOfSubList` that accepts another list *L* as a parameter and returns the starting index of where *L* first appears in this list, or `-1` if it is not found. All elements of *L* must appear in sequence and in the same order. For example, if variables called `list1` and `list2` store `[11, -7, 3, 42, 0, 14]` and `[3, 42, 0]`, respectively, the call of `list1.indexOfSubList(list2)` should return `2`.
3. Write a method called `replaceAll` that accepts two integer values as parameters and replaces all occurrences of the first value in the list with the second value. For example, if a variable called `list` stores `[11, -7, 3, 42, 3, 0, 14, 3]`, the call of `list.replaceAll(3, 999);` should change the list to store `[11, -7, 999, 42, 999, 0, 14, 999]`.

4. Write a method called `reverse` that reverses the order of the elements in the array list. For example, if a variable called `list` stores `[11, -7, 3, 42, 0, 14, 56]`, the call of `list.reverse();` should change the list to store `[56, 14, 0, 42, 3, -7, 11]`. An empty or one-element list is not changed by a call to this method.
5. Write a method called `runningTotal` that returns a new `ArrayList` that contains a running total of the original list. In other words, the i th value in the new list should store the sum of elements 0 through i of the original list. For example, given a variable `list` that stores `[2, 3, 5, 4, 7, 15, 20, 7]`, consider what happens when the following call is made:

```
ArrayList list2 = list.runningTotal();
```

The variable `list2` should store `[2, 5, 10, 14, 21, 36, 56, 63]`. The original list should not be changed by the method. If the original list is empty, the result should be an empty list. The new list should have the same capacity as the original. Remember that there is a list constructor that accepts a capacity as a parameter.

6. Write a method called `fill` that accepts an integer value as a parameter and replaces every value in the list with that value. For example, if a variable called `list` initially stores `[42, -7, 3, 0, 15]` and the call of `list.fill(2);` is made, the list will be changed to store `[2, 2, 2, 2, 2]`.

7. Write a method called `isPairwiseSorted` that returns whether or not a list of integers is pairwise sorted. A list is considered pairwise sorted if each successive pair of numbers is in nondecreasing order. For example, if a variable called `list` stores `[3, 8, 2, 5, 19, 24, -3, 0, 4, 4, 8, 205, 42]`, then the call of `list.isPairwiseSorted()` should return `true` because the successive pairs of this list are all sorted: `(3, 8), (2, 5), (19, 24), (-3, 0), (4, 4), (8, 205)`. The extra value `42` at the end had no effect on the result because it is not part of a pair. If the list had instead stored `[7, 42, 308, 409, 19, 17, 2]`, then the method should return `false` because the pair `(19, 17)` is not in sorted order. If a list is so short that it has no pairs, then it is considered to be pairwise sorted.
8. Write a method called `count` that accepts an element value as a parameter and returns the number of occurrences of that value in the list. For example, suppose a variable named `list` stores `[2, -3, 2, 0, 5, 2, 2, 6]`. A call of `list.count(2)` should return `4` because there are four occurrences of that value in the list.
9. Write a method called `maxCount` that returns the number of occurrences of the most frequently occurring value in a sorted list of integers. Because the list will be sorted, all duplicates will be grouped together, which will make it easier to count duplicates. For example, if a variable called `list` stores `[1, 3, 4, 7, 7, 7, 7, 9, 9, 11, 13, 14, 14, 14, 16, 16, 18, 19, 19, 19]`, the call of `list.maxCount()` should return `4` because

the most frequently occurring value (7) occurs four times. It is possible that there will be a tie for the most frequently occurring value, but that doesn't affect the outcome because you are just returning the count, not the value. If there are no duplicates in the list, then every value will occur exactly once and the max count is 1. If the list is empty, the method should return 0.

10. Write a method called `longestSortedSequence` that returns the length of the longest sorted sequence within a list of integers. For example, if a variable called `list` stores `[1, 3, 5, 2, 9, 7, -3, 0, 42, 308, 17]`, then the call of `list.longestSortedSequence()` would return 4 because it is the length of the longest sorted sequence within this list (the sequence `-3, 0, 42, 308`). If the list is empty, your method should return 0. Notice that for a nonempty list the method will always return a value of at least 1 because any individual element constitutes a sorted sequence.
11. Write a method called `removeLast` that removes and returns the last value from a list of integers. For example, if a variable called `list` stores `[8, 17, 42, 3, 8]`, a call of `list.removeLast();` should return 8 and change the list's state to `[8, 17, 42, 3]`. The next call would return 3 and remove 3 from the list, and so on. If the list is empty, throw a `NoSuchElementException`.
12. Write a method called `removeFront` that takes an integer n as a parameter and that removes the first n values from a list of

integers. For example, if a variable called `list` stores `[8, 17, 9, 24, 42, 3, 8]` and a call of `list.removeFront(4);` is made, the list's contents should become `[42, 3, 8]`. You may assume that the parameter value passed is between `0` and the size of the list inclusive.

13. Write a method `removeAll` that accepts an integer value as a parameter and that removes all occurrences of the given value from the list.
14. Write a method called `printInversions` that lists all inversions in a list of integers. An inversion is a pair of numbers in which the first appears before the second in the list, but the first is greater than the second. Thus, for a sorted list such as `[1, 2, 3, 4]` there are no inversions at all, and the method would produce no output. Suppose that a variable called `list` stores the values `[4, 3, 2, 1]`. The call of `list.printInversions();` would print many inversions:

```
(4, 3)
(4, 2)
(4, 1)
(3, 2)
(3, 1)
(2, 1)
```

The inversions can appear in any order, so this is just one possible correct output. You must reproduce this format

exactly, but the inversions can appear in any order. You may assume that the list has no duplicates.

15. Write a method called `mirror` that doubles the size of a list by appending the mirror image of the original sequence to the end of the list. The mirror image is the same sequence of values in reverse order. For example, if a variable called `list` stores `[1, 3, 2, 7]` and the client calls `list.mirror();` then the list should be changed to store `[1, 3, 2, 7, 7, 2, 3, 1]`. Notice that it has doubled in size because the original sequence now appears in reverse order at the end of the list.
16. Write a method called `stutter` that replaces every value with two of that value. For example, if the list initially stores `[42, 7, 0, -3, 15]`, after the call it should store `[42, 42, 7, 7, 0, 0, -3, -3, 15, 15]`.
17. Write a method called `stretch` that takes an integer n as a parameter and that increases a list of integers by a factor of n by replacing each integer in the original list with n copies of that integer. For example, if a variable called `list` stores `[18, 7, 4, 24, 11]` and we make the call of `list.stretch(3);` the list should be changed to store `[18, 18, 18, 7, 7, 7, 4, 4, 4, 24, 24, 24, 11, 11, 11]`. If n is zero or negative, the list should become empty.
18. Write a method called `doubleList` that doubles the size of a list by appending a copy of the original sequence to the end of the list. For example, if the list stores `[1, 8, 2, 7]`, your method should change it to store `[1, 8, 2, 7, 1, 8, 2, 7]`.

19. Write a method called `compress` that replaces every pair of elements in the list with a single element equal to the sum of the pair. If the list is of odd size, leave the last element unchanged. For example, if the list stores `[1, 7, 3, 9, 4, 6, 5]`, your method should change it to store `[8, 12, 10, 5]` ($1 + 7$, then $3 + 9$, then $4 + 6$, then 5).
20. Write a method called `rotate` that moves the value at the front of a list of integers to the end of the list. For example, if a variable called `list` stores `[8, 23, 19, 7, 12, 4]`, the call of `list.rotate();` should move the value 8 from the front of the list to the back of the list, producing `[23, 19, 7, 12, 4, 8]`.
21. Write a method called `switchPairs` that switches the order of values in the list in a pairwise fashion. Your method should switch the order of the first two values, then switch the order of the next two, switch the order of the next two, and so on. If the list contains an odd number of values, the final element is not moved. For example, if the list initially stores `[10, 25, 31, 47, 52, 68, 77]`, your method should switch the first pair (10 and 25), the second pair (31 and 47), and the third pair (52 and 68) to yield `[25, 10, 47, 31, 68, 52, 77]`.
22. Write a method called `fromCounts` that converts an `ArrayList` of counts into a new `ArrayList` of values as follows. Assume that the list stores a sequence of integer pairs that each pair indicates a count and a number. For example, suppose that an `ArrayList` called `list` stores `[5, 2, 2, -5, 4, 3, 2, 4, 1, 1, 1, 0, 2, 17]`. We will interpret

this sequence of pairs to mean that you have 5 occurrences of 2, followed by two occurrences of -5 followed by 4 occurrences of 3, and so on. So the call of:

```
ArrayList list2 = list.fromCounts();
```

should cause the variable `list2` to store `[2, 2, 2, 2, 2, -5, -5, 3, 3, 3, 3, 4, 4, 1, 0, 17, 17]`.

Programming Projects

1. The actual `List` interface in the `java.util` package has several methods beyond the ones implemented in this chapter. Write a version of `ArrayList<E>` that adds some or all of these methods. The methods to add are as follows (some headers are slightly modified; see the Java API Specification for descriptions of each method):

- `public void addAll(int index, ArrayList<E> list)`
- `public boolean containsAll(ArrayList<E> list)`
- `public boolean equals(Object o)`
- `public int lastIndexOf(Object o)`
- `public boolean remove(Object o)`
- `public void removeAll(ArrayList<E> list)`
- `public void retainAll(ArrayList<E> list)`
- `public Object[] toArray()`

2. The `java.util` package has an interface called `ListIterator` that extends the `Iterator` interface and includes additional methods specific to iterating through the elements of lists forward or backward. Write a class called `ArrayListIterator` that adds some or all of these methods. The methods to add are as follows (see the Java API Specification for descriptions of each method):

- `public void add(E value)`
- `public boolean hasPrevious()`
- `public int nextIndex()`
- `public E previous()`
- `public int previousIndex()`
- `public void set(E value)`

3. The actual `ArrayList` class in the `java.util` package has a method called `subList` that returns a view of a subportion of a list through a given range of indexes. It can be useful to think of part of a list as if it were its own list, complete with its own set of indexes and values. The sublist is “backed” by the original list, meaning that it is not a copy; if any change is made to the sublist, the original list is also affected. In order to implement this method, you will need to write an inner class inside `ArrayList<E>` that extends `ArrayList` and implements the behavior of the sublist. Override the methods for getting and setting values at particular indexes, as well as the `size` method, so that they reflect the sublist’s index range and size. Also, modify the outer `ArrayList<E>` class so that it always refers to its own elements through the use of these methods. The outer class should be given the following new method that returns an object of your new inner sublist class:

```
public ArrayList<E> subList(int fromIndex, int toIndex)
```

4. Based on the implementation of `ArrayIntList` or `ArrayList`, write a class `SortedIntList` or `SortedList` that provides most of the same operations but maintains its elements in sorted order. When a new value is added to the sorted list, rather than appending it to the end of the list, it is placed in the appropriate index to maintain sorted order of the overall list. For efficiency, you should discover the appropriate place to add new values to the list by using a binary search. Shift elements as needed and add the element in the proper index to maintain sorted order. (Do not manually re-sort the elements such as by calling `Arrays.sort`.) You should also modify the class's `indexOf` method to use a binary search to locate elements.

Since the list must remain sorted, your sorted list should not retain the following operations from `ArrayIntList` or

`ArrayList`:

```
public void add(int index, int value)
public void set(int index, int value)
```